



VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

Introduction to Software Engineering

Software Engineering

KIEU Quoc Viet HUYNH Vinh Nam

ICT Department, USTH



Agenda

- 1 Introduction
- 2 What is Software Engineering?
- 3 Why Learn Software Engineering?

1. Introduction

Software Engineering Introduction

Our Civilization Runs on Software!



Software Engineering Introduction

Our Civilization Runs on Software!



Software Engineering Introduction

Our Civilization Runs on Software!



Software Engineering Introduction

Our Civilization Runs on Software!



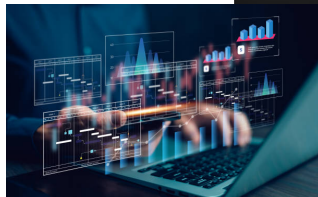
Software Engineering Introduction

Our Civilization Runs on Software!

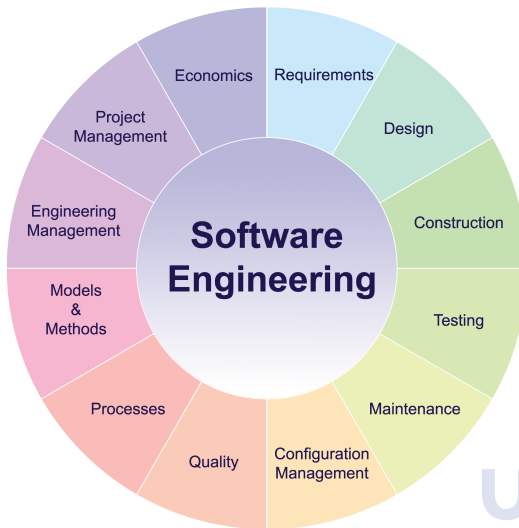


Software Engineering Introduction

Our Civilization Runs on Software!



Software Engineering Introduction



Questions to Consider

- ① Each software-related character's role
- ② What problems they may face, and how to solve them
- ③ What should we expect from them?
- ④ What they're expecting from us.
- ⑤ What are their best practices, tools, and techniques?
- ⑥ How do we know if they are doing a good job or not?
- ⑦ Can we measure the quality of their work?
- ⑧ The language they use everyday.



Questions to Consider (cont.)

① Each software-related character's role:

- Developer
- Project manager
- Product owner
- UX designex
- etc.

② What problems they may face:

- Communication
- Requirement analysis
- Project management
- BUGS, BUGS, and BUGS
- ▷ What's important, say it thrice.
- etc.

2. What is Software Engineering?

From Coder to Engineer



Translates a **clear solution** into code

Solves problems then codes. Knows algorithms & data structures

Defines the problem, designs the solution, coordinates team

Designs the **overall architecture**, works across the full system

	Coder	Programmer	Developer	Engineer
Writes code	✓	✓	✓	✓
Solves problems		✓	✓	✓
Defines problems			✓	✓
System architecture				✓

What is Software Engineering?

Operating
systems

Drivers

Embedded
systems

Compilers

Desktop

Web

Mobile

Database

Media

Games

and more

Definition

- 1 Understand the **context**
 - ▶ Pick the **right solution** for the context
- 2 Develop the **best solution** — effective & efficient
- 3 From **vague idea** → intuitive, scalable solution

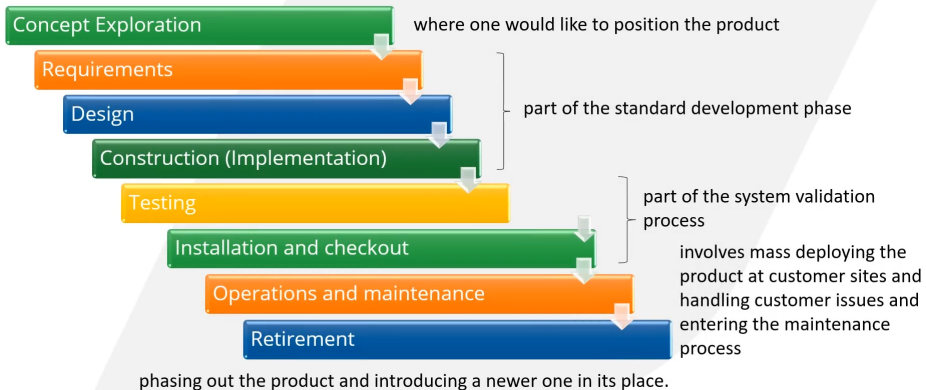
Benefits

- Apply **proven techniques** across tasks & projects
- Accurately predict **cost** & **schedule**
- Build desirable characteristics in the products (i.e: reliability, maintainability, and usability, etc.)

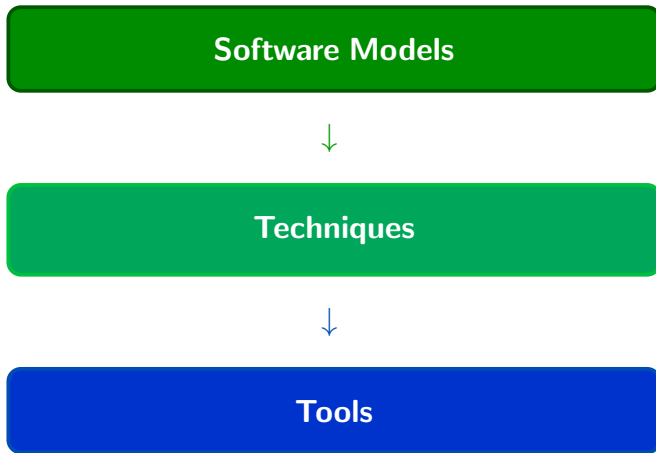
Differences between SE and other "E"

- Software is intangible, invisible, and non-physical
- Software is built to evolve
- Successful applications often continue to evolve long after the initial release

What is Software Engineering? (cont.)



What is Software Engineering? (cont.)



3. Why Learn Software Engineering?

Reasons for Learning Software Engineering

Learning new techniques, best practices, and new tools



More productive



More time



More software



More profits for you and your organization

Software Crisis

A term that talks about **MANY** difficulties (obviously 😊)

Software Crisis

A term that talks about **MANY** difficulties (obviously 😊)

- In developing large software systems
- During 60s and 70s
- Software failures occurs more frequently than they should (that's a fact 😊)

Software Crisis

A term that talks about **MANY** difficulties (obviously 😊)

- In developing large software systems
 - During 60s and 70s
 - Software failures occurs more frequently than they should (that's a fact 😊)
- ▷ Still valid today!

Software Crisis (cont.)

- 1 Never completed systems

Software Crisis (cont.)

- 1 Never completed systems
- 2 Missed deadlines

Software Crisis (cont.)

- 1 Never completed systems
- 2 Missed deadlines
- 3 Exceeded budgets

Software Crisis (cont.)

- 1 Never completed systems
- 2 Missed deadlines
- 3 Exceeded budgets
- 4 A system that does not do all that is required of it
- 5 A system that works but is difficult to use
- 6 A system is difficult to modify

Software Crisis (cont.)

- ① Never completed systems
- ② Missed deadlines
- ③ Exceeded budgets
- ④ A system that does not do all that is required of it
- ⑤ A system that works but is difficult to use
- ⑥ A system is difficult to modify
- ⑦ A loss of trust from users
- ⑧ etc.

Current Challenges in Software Engineering

- The need to develop trustworthy high-quality software
- The higher demand of quick turnaround from concept to deployment and operation
- Increased software complexity
- The diversity of software systems

Current Challenges in Software Engineering

- The need to develop trustworthy high-quality software
- The higher demand of quick turnaround from concept to deployment and operation
- Increased software complexity
- The diversity of software systems
- **HANDLE CHANGE.**

Why Learn Software Engineering?

Many project failures result from:

- The inability to scale the techniques employed.
- The lack of any framework for planning and organization software development projects.
- etc.

Why learn Software Engineering (cont.)

Why learn Software Engineering (cont.)

The whole team approach concept!

Why learn Software Engineering (cont.)

The whole team approach concept:

- emphasizes the notion of generalizing specialists

Why learn Software Engineering (cont.)

The whole team approach concept:

- emphasizes the notion of generalizing specialists

Note:

- Generalizing specialists with cross-functional skills can perform many different tasks on projects
- People who can act in more than one role are highly preferred.
- Any team member might complete any task. Anyone can almost do anything.

Why learn Software Engineering (cont.)

The whole team approach concept:

- emphasizes the notion of generalizing specialists

Note:

- Generalizing specialists with cross-functional skills can perform many different tasks on projects
- People who can act in more than one role are highly preferred.
- Any team member might complete any task. Anyone can almost do anything.
 - The whole team can work together to come up with the best design for the software
 - The whole team is self-managing!

- involve constant collaboration!

Summary

- ① Structured approach for large/complex systems
- ② Problem decomposition
- ③ Shared understanding & communication
- ④ Extensive group work
- ⑤ Clear task ownership & interfaces
- ⑥ Common language across teams